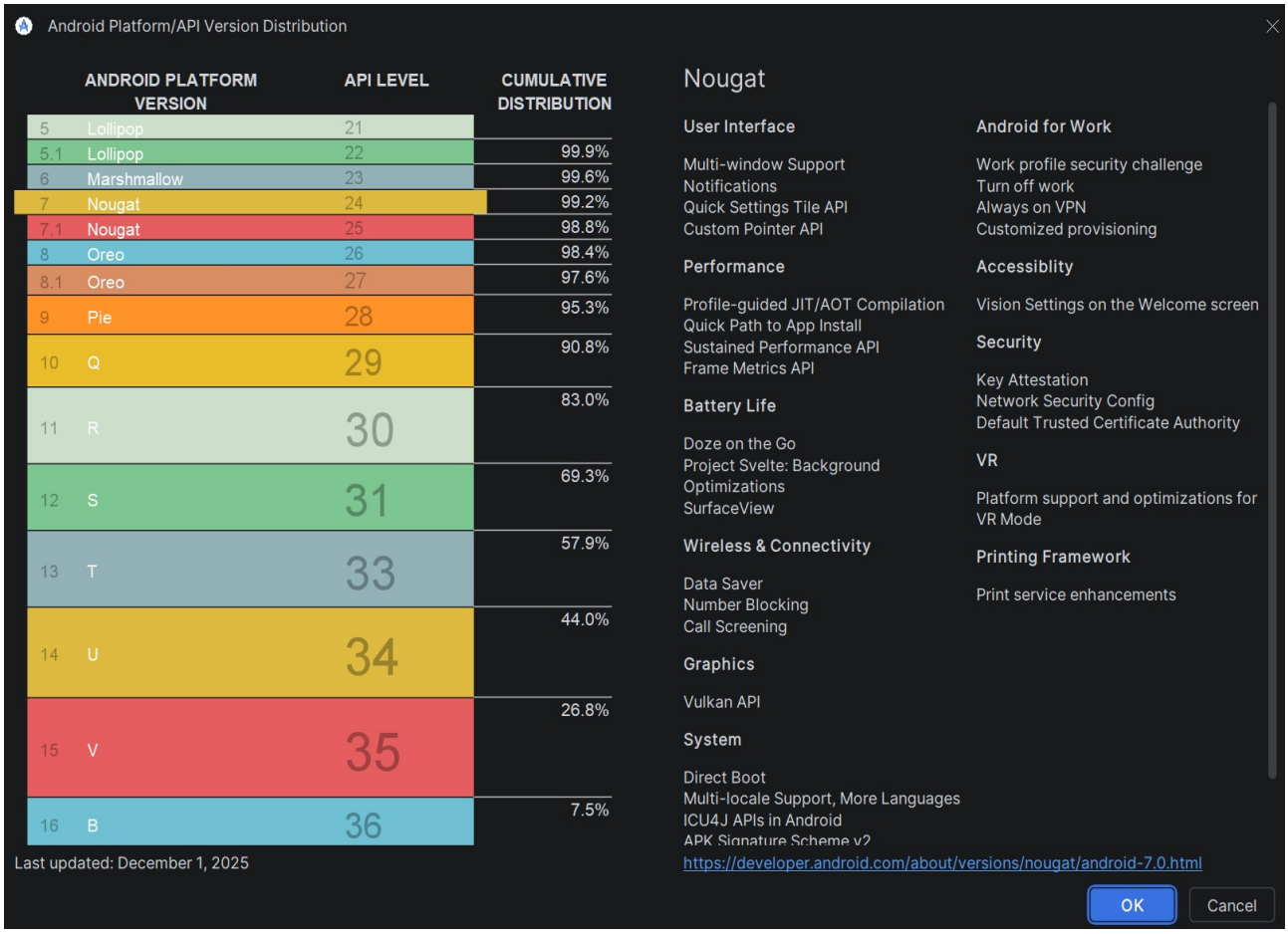
¡Hola mundo!, de Android

En esta actividad vas a crear la primera aplicación nativa en Android.

Para crear un nuevo proyecto en Java debes seleccionar **No Activity** introducir el nombre del proyecto (Hello World) nombre del paquete de tu aplicación (com.example.helloworld) y seleccionar el Languaje **Java**. A continuación se deben definir los dispositivos destino, seleccionando el nivel más bajo de API para el que funcoinará esta aplicación. Es recomendable seleccionar API 24 (“Nougat”; Andorid 7.0), con lo que funcionará en un 99.2% de dispositivos.



Si se han seguido los pasos anteriores, el entorno habrá generado todos los elementos necesarios de la aplicación, las carpetas estudiadas anteriormente, con las librerías y referencias necesarias para la compilación y ejecución de la aplicación. También estará listo el *AndroidManifest.xml*.

Para crear nuestra primera pantalla en la aplicación debemos crear una clase Java que contenga nuestro ***MainActivity***. En la carpeta **kotlin+java/com.example.helloworld** crea una nueva **Java Class** y llámala ***MainActivity***. Esta será la clase que contendrá el Activity de nuestro proyecto donde mostraremos el texto “Hello World!”.

Antes de escribir la clase ***MainActivity*** tenemos que crear el XML que contendrá la vista de la actividad. Para ello en la carpeta ***res*** creamos una subcarpeta ***layout*** y dentro de ***res/layout*** un archivo ***activity_main.xml***

En el archivo ***activity_main.xml*** puedes copiar el siguiente contenido:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="hello_world!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

El contenido del XML lo estudiaremos más adelante, pero ahora basta con conecer que consta de un contenedor (en este caso, un ConstraintLayout) y un objeto de texto en su interior (TextView); este texto es el que ha apaercido en pantalla. Además, también vienen definidos la codificación y el esquema de Android que le permit reconocer los comandos básicos

Ahora podemos volver al fichero ***MainActivity.java*** y completarlo con el siguiente código:

```
package com.example.emptyviewactivity;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

Vamos a analizar este código, ya que segirá un modelo de uso común en la programación en Android. Lo primero que aparece es el paquete en el que irá envuelta la aplicación que, en este caso es una simple clase Java, pero con la peculiaridad de que hereda (extends)de la superclase AppCompatActivity, lo que proporciona a esta clase la posibilidad de “entender” el código específico de Android.

Por tanto, se puede entender que *import androidx.appcompat.app.AppCompatActivity;* uno de los import que precede a la clase, facilita a la aplicación las librerías que contien esta superclase.

Dentro de la clase MainActivity, tan solo se incluye un método, sobreescrito (@Override), que recordando lo visto en la teoría, responderá en el momento que la aplicaciónse cree y entre en funcionamiento en la memoria del teléfono.

En el momento que se crea la aplicación, ocurren dos hechos importantes. Por un lado, se prepara un Bundle (paquete de almacenaje) donde guardar las instancias y esdos de la aplicación para ser recuperados en caso de necesisda. Para ello también existe una librería (import) que suministra este paquete.

El otro hecho importante que ocurre, cuando se crea la actividad, viene marcado con la líne de código *setContentView(R.layout.activity_main)*. Esta lo que hace es asociar el fichero de la vista a la clase Java, utilizando para ello la clase R como gestora. Y tan solo con eso se “inflará” el contenido visual definido dentro del fichero XML antes visto.

Estos dos puntos son principios muy elementales, pero imprescindibles para dar lso primeros pasos en el conocimiento e Android.

Si ahora ejecutas tu aplicación verás que da error, esto es debido a que no has declarado la actividad en el Manifest. Debes hacerlo escribiendo el siguiente código antes de la etiqueta *</application>*

```
<activity
    android:name=".MainActivity"
    android:exported="true"
    android:windowSoftInputMode="adjustResize">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

Ahora si puedes ejecutar tu aplicación en el emulador o dispositivo conectado que hayas configurado.

Ejercicio 1

Si se añade el siguiente código a la aplicación, se consigue un Log informativo que avisa del paso por el momento del ciclo de vida Started (para que reconozca el comando, Android Studio importá la librería android.util.Log)

Es recomendable filtrar en el entorno de desarrollo, los Log por tipo (Verbose, Debug, Info, Warn, Error, Assert), además de establecer un filtro personalizado por Tag (en este caso el Tag definido es “EJEMPLO”).

```
@Override
protected void onStart() {
    super.onStart();
    Log.i("Ejemplo", "Estoy onStarted");
}
```

Siguiendo el ejemplo anterior, implementa Logs en tu aplicación que informen del paso por cada uno de los estados posibles del ciclo de vida de la aplicación que hemos visto en la teoría.

Ejercicio 2

Como hemos visto en clase, un Intent es un sistema de comunicación que permite interactuar entre componentes de la misma o distintas aplicaciones de Android. Con él se puede comenzar una aplicación, iniciar un servicio o entregar un mensaje. También hemos visto que hay dos tipos de Intents: explícito e implícitos.

Los Intents explícitos son aquellos que nombran el componente que se necesita ejecutar, la clase Java específica que se necesita para realizar alguna tarea. En este ejercicio vamos a programar un ejemplo que hace uso de este tipo de Intent.

Crea una segunda actividad llamada ***SecondActivity***. Crea también si fichero vista XML y llámalo ***activity_second.xml***. Ahora asocia en la segunda actividad la vista a este fichero. El código de esta debe quedarte así:

```
public class SecondActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
    }
}
```

Retoca el objeto de texto del Segundo XML y sustituye el “Hello World!” por otra cadena distinta. A continuación añade el siguiente código al método ***onDestroy*** de la clase ***MainActivity***

```
Intent ejemplo = new Intent(this, MainActivity.class);
startActivity(ejemplo);
```

Recuerda que debes añadir el activity que hemos creado al Manifest. Puedes hacerlo escribiendo el siguiente código despues de la etiqueta de cierre (</activity>) de la primera actividad, dentro del Manifest.

```
<activity android:name=".SecondActivity"
</activity>
```

Si ahora ejecutas y pulsas el botón de Atrás del dispositivo verás que se cierra la MainActivity y, posteriormente, se abre la SecondActivity.

Ejercicio 3

Los Intent implícitos por otro lado son aquellos con los que tan solo informamos la acción que deseamos realizar, y el sistema nos responde con el componente más adecuado para realizar dicha petición, es decir no se especifica un componente concreto. En este ejercicio vamos a ver una página web, y será el sistema el que propondrá el navegador con el que hacerlo.

Añade el siguiente código al método onDestroy de la clase SecondActiviy. Verás que cuando eliminas esta se te abre el navegador con la página de Google (para cargar las libresrías tan solo debes posiconarte en los elementos desconocido y pulsar Alt+Intro).

```java
@Override
protected void onDestroy() {
    super.onDestroy();
    Intent ejemplo = new Intent(Intent.ACTION_VIEW);
    ejemplo.setData(Uri.parse("http://www.google.es"));
    startActivity(ejemplo);
}
```